

Case study: Arm and x86

COMS10015

Dr Tom Deakin

University of Bristol

Week 15

Arm

Arm Holdings plc

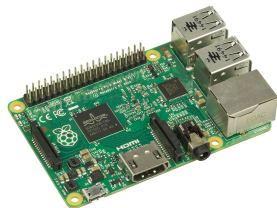
- ▶ Founded in 1990, based in Cambridge, UK, mostly owned by SoftBank Group (Japanese investment company)¹.
- ▶ Arm design instruction sets for CPUs (Cortex, SecurCore, Neoverse), GPUs (Immortalis, Mali), and NPUs (Ethos), and core-to-core interconnects.
- ▶ Provide physical micro-architecture designs for implementations of the ISA for automotive, consumer and infrastructure (server).
- ▶ System designs (e.g., CoreLink Interconnect) for building a whole processor from cores.
- ▶ They *do not* manufacture any processors, they licence the Intellectual Property (IP).
- ▶ The most **widely used** ISA today, with over 200 Billion chips shipped (as of 2021²).

¹https://en.wikipedia.org/wiki/Arm_Holdings

²<https://newsroom.arm.com/blog/200bn-arm-chips>

ARMv7-A

- ▶ ARMv7-A is one of the ISAs from Arm.
- ▶ Has a Reduced Instruction Set Computer (RISC) design.
- ▶ Load-store architecture (operations happen on registers, not directly on memory).
- ▶ 32-bit instruction set (word size).



Raspberry Pi 2
micro-computer with
Arm Cortex-A7 processor
from Broadcom
Image from Wikipedia

ARMv7-A: state: registers

General Purpose Registers

- ▶ 16 general purpose 32-bit registers: R0-R15.
- ▶ **But**, three have special usage:
 - R13/SP stack pointer
 - R14/LR link register
 - R15/PC program counter
- ▶ Use of the SP and PC as a general purpose register is deprecated (i.e., don't do it!)
- ▶ “When executing an ARM instruction, PC reads as the address of the current instruction plus 8”³ — a 2 instruction offset⁴

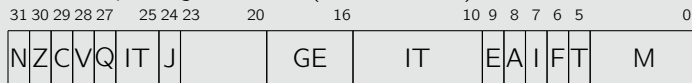
³ ARM Architecture Reference Manual: ARMv7-A and ARMv7-R edition: Chapter A2.3

⁴ Why? Implementation simplification for a pipelined processor

ARMv7-A: state: registers

Special Purpose Registers

- ▶ 2 special purpose 32-bit registers: CPSR and SPSR.
- ▶ CPSR is available to applications (through an APSR alias).
- ▶ SPSR is available in privileged modes (see Week 21).



N,Z,C,V condition flags for negative, zero, carry, overflow after instruction execution.

E Endianness state for load/stores.

A,I,F bits used during interrupt handling.

M processor mode.

The `mrs r0, cpsr` instruction copies values into a GPR.

The `msr cpsr, r0` copies (bits) from a GPR (also accepts an immediate).

ARMv7-A: state: memory

Memory

- ▶ Memory space of 2^{32} 8-bit bytes (byte-addressable) — total capacity of 4 GB.
- ▶ Also supports access as 2^{30} 32-bit words (word-aligned access), and 2^{31} 16-bit halfwords (halfword-aligned access).
- ▶ Could be big- or little-endian

Recall Lecture on Architecture and memory space (granularity).

ARM Architecture Reference Manual: ARMv7-A and ARMv7-R edition: Chapter A1.5 and A3.1.

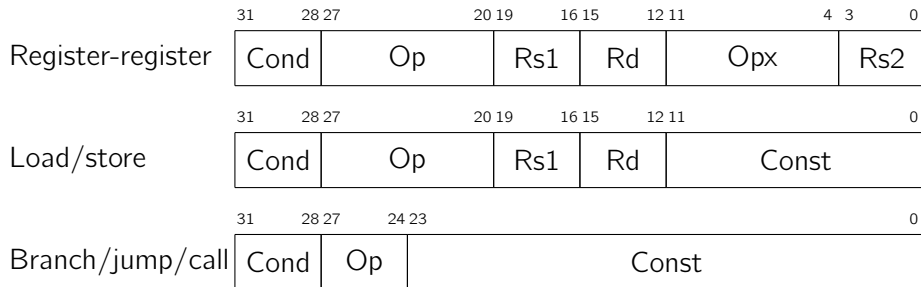
Addressing modes

Name	Example
Immediate	MOV R0, #15
Direct	LDR R0, MEM or MOV R0, R1
Indirect	LDR R0, [R1]
Indirect with offset	LDR R0, [R1, #4]
Indirect with pre-increment	LDR R0, [R1, #4]! (similar to ++i)
Indirect with post-increment	LDR R0, [R1], #4 (similar to i++)
Indirect indexed	LDR R0, [R1, R2]
Indirect indexed with scaling	LDR R0, [R1, R2, LSL #2]
PC relative	LDR R0, [PC, #offset]

ARM Architecture Reference Manual: ARMv7-A and ARMv7-R edition: Chapter A4.6.5

<https://www.labs.cs.uregina.ca/301/ARM-addressing/lecture.html>

Instruction encoding



Adapted from Patterson and Hennessy, Computer Organization and Design MIPS edition, p154

ARM Architecture Reference Manual: ARMv7-A and ARMv7-R edition: Chapter A5.2

Predicated execution

- ▶ Notice how each instruction has a `cond` prefix.
- ▶ Also, the instruction set contains **NO** conditional branches. . .

Predicated execution

- ▶ Notice how each instruction has a `cond` prefix.
- ▶ Also, the instruction set contains **NO** conditional branches. . .
- ▶ Arm instructions are executed conditionally, based on the value of the prefix and the CPSR special-purpose register.
- ▶ To do a conditional branch, first the comparison operations update the CPSR register (e.g., with `cmp`), then the branch instruction is *conditionally* executed based on the result.
- ▶ If the condition fails, then the instruction doesn't have an effect (NOP).

Predicated execution

- ▶ Notice how each instruction has a `cond` prefix.
- ▶ Also, the instruction set contains **NO** conditional branches. . .
- ▶ Arm instructions are executed conditionally, based on the value of the prefix and the CPSR special-purpose register.
- ▶ To do a conditional branch, first the comparison operations update the CPSR register (e.g., with `cmp`), then the branch instruction is *conditionally* executed based on the result.
- ▶ If the condition fails, then the instruction doesn't have an effect (NOP).

Some example conditions

Cond	Meaning	Condition
0000	Equal	$Z == 1$
0001	Not-equal	$Z == 0$
1100	Signed greater than	$Z == 0$ and $N == V$
1110	Always	N/A

x86

x86 ISA

- ▶ x86 evolved and developed by two different companies: Intel and AMD.
- ▶ Intel's 8086 processor in 1978 was the start, along with successor designs.
- ▶ 8086 had just one general purpose register (accumulator architecture).
- ▶ Each new processor added in new functionality, like floating-point, memory protection, SIMD vector extensions, etc.
- ▶ First had general purpose registers in 80386 in 1985.
- ▶ AMD increased address space to 64-bits in 2003 (AMD64).
- ▶ As of 2018, x86 has around 1,400 instructions⁵.
- ▶ Despite the complexity, most commonly used instructions are “easy” to implement⁶.
- ▶ In 2024, Intel and AMD and others start x86 Ecosystem Advisory Group ⁷.

⁵Patterson and Hennessy, Computer Organization and Design, Figure 2.46

⁶ibid

⁷[https:](https://www.intel.com/content/www/us/en/newsroom/news/october-2024-intel-news.html)

[//www.intel.com/content/www/us/en/newsroom/news/october-2024-intel-news.html](https://www.intel.com/content/www/us/en/newsroom/news/october-2024-intel-news.html)

IA-32

- ▶ IA-32 is the Intel 32-bit x86 ISA from the 80386 processor in 1985.
- ▶ Has a Complex Instruction Set Computer (CISC) design.
- ▶ Instructions have a variable-length encoding (recall Week 13).
- ▶ Register-memory architecture: instruction operands can be registers and/or memory addresses.
- ▶ Instructions operate on 8-bit bytes and either 16-bit words or 32-bit doublewords.
 - ▶ Use a bit in one of the segment registers to choose the default word size.
 - ▶ Prefix added to instruction to override.

IA-32: state

- ▶ (Virtual) Address space of 2^{32} bytes (4 GB), and a physical address space of 2^{36} bytes (64 GB).
- ▶ 8 general-purpose 32-bit registers, each with a particular name (EAX, ECX, EDS, EBX, ESP, EBP, ESI, EDI).
- ▶ ESP has a “special” use to hold the stack pointer.
- ▶ Instructions often use specific registers for operands (i.e., not completely “general purpose”).
- ▶ Also 6 16-bit segment registers to access segmented memory (we won’t really cover what that is in this course).
- ▶ Two special-purpose 32-bit registers: EFLAGS to hold status flags, and EIP for instruction pointer (address of next instruction to execute).

IA-32: control flow

- ▶ Conditional branch instructions use the status register to determine if branch is taken, rather than an operand.
 - ▶ E.g., `cmp eax, ebx` sets the status flags based on the result of the comparison.
 - ▶ Then `jnz offset` jumps if $ZF == 0$ in EFLAGS.
 - ▶ The offset here must be in bytes because not all instructions are the same length!
- ▶ Note this is **not at all** predicated execution, which in Armv7 is applied to all instructions.

What does this x86 program do?

```
.loop:  
    xadd    rax,rdx  
    loop    .loop
```

What does this x86 program do?

```
.loop:  
    xadd    rax,rdx  
    loop    .loop
```

Need some instruction definitions!

xadd Rd, Rs Exchange and add: Rd value stored in Rs, and Rd+Rs stored in Rd.

loop .label Performs loop using ECX register as a counter. ECX decremented and if not zero, jump to operand (as offset to EIP).

Example from xchg rax,rax, <https://www.xorpd.net>

What does this x86 program do?

The two registers take the following values:

rax	rdx
-----	-----

0	1
---	---

1	0
---	---

1	1
---	---

2	1
---	---

3	2
---	---

5	3
---	---

8	5
---	---

13	8
----	---

... // until ECX is zero

What do you notice about rax?

What does this x86 program do?

The two registers take the following values:

rax	rdx
-----	-----

0	1
---	---

1	0
---	---

1	1
---	---

2	1
---	---

3	2
---	---

5	3
---	---

8	5
---	---

13	8
----	---

... // until ECX is zero

What do you notice about rax?

The instruction sequence sets rax to the numbers in the Fibonacci sequence.

Example from `xchg rax,rax`, <https://www.xorpd.net>

Further Reading

- ▶ ARM Architecture Reference Manual ARMv7-A and ARMv7-R edition, <https://developer.arm.com/documentation/ddi0406/cd/?lang=en>
- ▶ <https://stackoverflow.com/questions/24091566/why-does-the-arm-pc-register-point-to-the-instruction-after-the-next-one-to-be-e>
- ▶ <https://en.wikipedia.org/wiki/X86>
- ▶ Intel 64 and IA-32 Architectures Software Developer's Manual, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- ▶ Patterson and Hennessey, Computer Organization and Design, MIPS edition, Chapter 2.16–2.19.
- ▶ Patterson and Hennessey, Computer Organization and Design, MIPS edition, 2.23 (available online), <https://www.elsevier.com/books-and-journals/book-companion/9780128201091>.